

WorldMemoryLib

Cahier des Charges Technique

Librairie Minecraft de Versionnage et Rollback du Monde

Version	0.1 — Conception
Date	09/04/2026
Statut	Conception — Pré-développement
Loader cible	À définir (Fabric / NeoForge / Architecture)
Langage	Java 17+

Document issu d'une session de conception approfondie. Il synthétise l'ensemble des décisions architecturales prises avant le démarrage du développement.

1. Vision du Projet

WorldMemoryLib est une librairie Minecraft open-source qui fournit un système de versionnage et de rollback du monde inspiré de Git. Elle permet à n'importe quel mod de capturer, stocker et restaurer des états du monde de façon efficace, granulaire et configurable.

Le projet est né du besoin d'un mod de magie du temps nécessitant la capacité de restaurer des zones détruites par des explosions cataclysmiques, tout en respectant les constructions des joueurs et en gérant l'historique complet des états du monde.

1.1 Philosophie de Design

- Aussi cruel que miséricordieux : la lib ne prend pas de décision éthique, elle expose des outils.
- Efficacité avant tout : inspirée de Git, seules les différences sont stockées.
- Extensible : les mods tiers peuvent enregistrer leurs propres entités explosives.
- Transparente en cas de dégradation : un rollback approximatif vaut mieux qu'une absence de rollback.

1.2 Cas d'Usage Principaux

Cas	Description	Mode
Restauration bienveillante	Réparer une base détruite par un Wither	DELTA_EXPLOSIONS_ONLY
Oubli Total (cruel)	Effacer toute présence humaine dans une zone	SEED_ORIGINAL
Chirurgie temporelle	Restaurer une structure précise sans toucher au reste	BLOC ciblé
Anti-grief admin	Rollback d'une grieffe TNT massive	ETAT_PRECEDENT
Reset conditionnel	Purge automatique après X jours d'inactivité	PURGE_CONDITIONNELLE

2. Architecture Générale

La lib est organisée en quatre couches indépendantes et empilées :

COUCHE 4 — PURGE	Normale / Administrative / Conditionnelle
COUCHE 3 — ROLLBACK	Granularité Macro→Micro, Modes, Cohérence
COUCHE 2 — HISTORIQUE	Stack temporel, Causes, Flags, Métadonnées
COUCHE 1 — STOCKAGE GIT-LIKE	Hash SHA-1, Compression, Delta vs Seed

■ Chaque couche peut être utilisée indépendamment. Un mod peut n'utiliser que le stockage Git-like sans le système de rollback complet.

3. Couche 1 — Stockage Git-like

3.1 Principes Fondamentaux

- Content Addressable Storage (CAS) : chaque chunk est identifié par le hash SHA-1 de son contenu.
- Déduplication automatique : un chunk identique stocké une seule fois, peu importe combien de snapshots le référencent.
- La seed comme commit zéro gratuit : aucun chunk non modifié n'est stocké, la seed suffit.
- Compression GZIP immédiate + optimisation ZSTD en arrière-plan.

3.2 Système de Commit Intelligent

Deux triggers de commit coexistent en permanence :

Trigger	Condition	Comportement
Temporel	Toutes les X minutes (configurable)	Commit automatique de l'état actuel
Seuil	Plus de 50 blocs modifiés en 5 minutes	Commit immédiat + reset du compteur
Explosion (prioritaire)	Entité explosive flaggée détectée	Pre-snapshot immédiat avant l'explosion

3.3 Filtre vs Seed

Après chaque capture, chaque bloc est comparé à son état de génération :

- avant == seed ET après == seed → ignoré, rien à stocker
- avant == seed ET après != seed → cratère dans terrain vierge, stocker seulement l'après
- avant != seed → construction joueur impliquée, stocker les deux états

■ Les chunks vierges non modifiés par l'explosion ne sont jamais stockés. La seed est toujours le plancher absolu implicite et indestructible.

4. Couche 2 — Historique

4.1 Stack Temporel par Chunk

Chaque chunk possède une pile (stack) d'états ordonnés dans le temps. La seed constitue le plancher absolu de la pile, implicite et jamais supprimé.

```
ChunkHistorique { ChunkPos position LinkedList<ChunkEtat> stack ChunkEtat { long timestamp String hash // référence vers l'objet store CauseModification cause UUID responsable boolean estPurgeable } }
```

4.2 Système de Causes

Chaque modification est trackée avec sa cause et une chaîne de responsabilité :

Cause	Description	Impact rollback
JOUEUR	Action volontaire d'un joueur	Ne jamais supprimer sans mode CRUEL
EXPLOSION_TNT	TNT primée	Restaurable par rollback bienveillant
EXPLOSION_CREEPER	Creeper normal ou chargé	Restaurable
EXPLOSION_CRYSTAL	End Crystal	Restaurable
EXPLOSION_WITHER	Wither skull ou invocation	Restaurable
GENERATION	Génération naturelle du monde	Référencé via seed
INCONNU	Source non identifiée	Traité comme JOUEUR par sécurité

4.3 Entités Explosives Flaggées

Le flag d'une entité explosive déclenche le pre-snapshot avant l'explosion. Un scan de propagation est lancé récursivement pour trouver toutes les TNT en chaîne avant de prendre un unique snapshot groupé.

Entité	Puissance	Rayon (blocs)	Délai
TNT	4.0	~8	4 secondes (80 ticks)
Creeper	3.0	~6	~1.5 secondes
Creeper chargé	6.0	~12	~1.5 secondes
End Crystal	6.0	~12	Immédiat
Wither (invocation)	7.0	~14	Immédiat
Wither Skull	1.0	~2	Immédiat
Lit (Nether/End)	5.0	~10	Immédiat

Respawn Anchor	5.0	~10	Immédiat
----------------	-----	-----	----------

4.4 Propagation en Graphe (Chaîne de TNT)

Algorithme de scan récursif déclenché à la détection d'une première entité explosive :

- Scan des blocs TNT dans le rayon d'effet → seront primés par l'explosion
- Scan des Primed TNT déjà actives dans le rayon → récupérer leur fuse restant
- Répéter pour chaque nouvelle TNT trouvée jusqu'à fermeture du graphe
- Un seul pre-snapshot de l'union de toutes les zones affectées
- Attendre fuse_max + 5 ticks de marge → un seul post-snapshot

■ *Anti-spam : le scan de propagation ne peut pas être déclenché plus de X fois par seconde (configurable). Évite le lag sur les serveurs avec redstone automatisée.*

5. Couche 3 — Rollback

5.1 Hiérarchie de Granularité

Le rollback est organisé en niveaux du plus macro au plus micro. Un rollback d'un niveau supérieur implique automatiquement les niveaux inférieurs dans la même zone, sauf exclusion explicite.

NIVEAU 1 — CHUNK	Le plus brutal. Restaure un chunk entier.	<i>SEED_ORIGINAL, ETAT_PRECEDENT, DELTA_EXPLOSIONS_ONLY</i>
NIVEAU 2 — BLOC	Chirurgical. Restaure des blocs précis ou une structure.	<i>Pos individuelle, rayon personnalisé</i>
NIVEAU 3 — ENTITÉ	Mobs, items au sol, projectiles.	<i>IGNORER, RESTAURER_MOBS, RESTAURER_TOUT, EFFACER_JOUEURS</i>
NIVEAU 4 — ITEM	Contenu des inventaires et items au sol.	<i>RESTAURER_CONTENEURS, RESTAURER_ITEMS_SOL</i>
NIVEAU 5 — NBT	Mémoire interne de chaque objet/entité.	<i>COMPLET, PARTIEL, AUCUN</i>

5.2 Modes de Rollback

Mode	Description	Constructions joueurs
SEED_ORIGINAL	Remet le chunk exactement comme à la génération	Supprimées (acte cruel)
ETAT_PRECEDENT	Remet le dernier snapshot connu avant l'événement	Préservées
DELTA_EXPLOSIONS_ONLY	Restaure uniquement ce que l'explosion a détruit	Intégralement préservées

5.3 Cas Spéciaux

Coffres — Principe de Schrödinger

Si le coffre n'avait jamais été ouvert avant l'explosion (tag LootTable présent, tag Items absent), la loot table est réassignée telle quelle. Le contenu sera généré à la prochaine ouverture comme si de rien n'était. Si le coffre avait été ouvert, le NBT complet est restauré.

Spawnpoints Joueurs

Les spawnpoints liés à des lits ou ancrs de respawn dans la zone sont trackés. Si le bloc de spawn est restauré par le rollback, le spawnpoint du joueur concerné est automatiquement réactivé.

Entités — Capture NBT avant disparition UUID

Les mobs tués dans la zone d'explosion sont capturés via leur NBT complet avant la disparition de leur UUID dans le cycle de suppression. La restauration resupmon l'entité à ses coordonnées

originales avec type, HP, équipement (main, offhand, armure) et NBT additionnel.

Portails Inter-Dimensions

Les blocs de portail sont restaurés avec leur NBT. La relation des portails est laissée à la charge du joueur ou du mod consommateur. La lib expose un événement onPortalRestored pour permettre au mod de gérer ce cas.

Zone Gelée Pendant le Cast

Pendant la durée d'un sort de rollback, la zone cible peut être gelée (aucune modification acceptée). Ce comportement est configurable : FREEZE_ZONE, NO_WARNING ou CANCEL_IF_MODIFIED.

6. Couche 4 — Purge

6.1 Types de Purge

Type	Description	Irréversible ?	Accès
AUCUNE	Historique conservé indéfiniment	Non	Tous
ADMINISTRATIVE	Purge manuelle par un admin. Audit log obligatoire avant suppression.	Oui	Admin uniquement
CONDITIONNELLE	Règles automatiques composables (âge, cause, event, inactivité...)	Configurable	Config serveur

6.2 La Seed comme Plancher Absolu

La seed n'est jamais stockée dans la pile des états — elle est implicite et indestructible. Même après une purge administrative complète, le chunk peut toujours être rollback à son état de génération. Il n'existe aucun moyen de supprimer ce plancher.

6.3 Politiques Conditionnelles

Les politiques de purge sont composables par opérateurs logiques :

- `parAge(joursMax)` — purger les états plus vieux que X jours
- `parCause(cause)` — purger uniquement les états d'une cause donnée
- `parInactivite(jours)` — chunk non chargé depuis X jours
- `parEvent(event)` — un événement magique déclenche la purge
- `policy1.et(policy2) / policy1.ou(policy2)` — composition

7. Gestion des Crashes

7.1 États d'une Chaîne d'Explosions

```
EN_ATTENTE // Graphe calculé, pre-snapshot pris, explosions pas encore commencées
EN_COURS // Explosions en cours – état instable TERMINEE // Post-snapshot pris –
rollback possible CRASH_DETECTE // Serveur crashé pendant EN_COURS
```

7.2 Comportement au Redémarrage

- Le serveur détecte les chaînes en état CRASH_DETECTE au démarrage.
- Si l'explosion est terminée : post-snapshot pris immédiatement (potentiellement contaminé, marqué POST_CRASH).
- Si des Primed TNT sont encore actives : le tracking reprend normalement.
- Le mod consommateur reçoit un flag estFiable=false et peut adapter son comportement (animation glitchée, message narratif...).

■ *Un rollback POST_CRASH est approximatif mais préférable à l'absence de rollback. La lib ne cache pas cette dégradation : elle l'expose clairement au mod consommateur qui peut en faire une feature narrative (mémoire temporelle fragmentée, etc.).*

8. Throttling et Performance

8.1 Groupage d'Explosions

Les explosions proches dans le temps (fenêtre configurable, défaut 500ms) et l'espace (rayon de fusion configurable, défaut 16 blocs) sont regroupées en un seul événement. L'union des blocs affectés est calculée une seule fois, éliminant toute redondance.

8.2 Priorité de Sauvegarde

Quand les ressources sont limitées, l'ordre de priorité est le suivant :

Priorité	Problème	Solution retenue
P1 — CRITIQUE	Constructions joueurs	Toujours sauvegardées en premier — perte irremplaçable
P2 — ÉLEVÉE	Block Entities (coffres, machines...)	NBT important — sauvegardé en second
P3 — NORMALE	Terrain modifié par explosion	Sauvegardé si ressources disponibles
P4 — BASSE	Terrain vierge dans le rayon	Jamais stocké — la seed suffit

8.3 Seuils Configurables

Paramètre	Défaut	Description
MAX_EXPLOSIONS_PAR_SECONDE	10	Au-delà → mode snapshot léger (métadonnées uniquement)
MAX_BLOCS_PAR_SNAPSHOT	10 000	Au-delà → snapshot partiel par priorité
MAX_SNAPSHOTS_EN_MEMOIRE	50	Au-delà → refus avec log d'avertissement
SCAN_COOLDOWN_MS	200ms	Délai minimum entre deux scans de propagation
FENETRE_GROUPEMENT_MS	500ms	Fenêtre de fusion des explosions proches
RAYON_FUSION_BLOCS	16	Rayon de fusion spatiale des explosions
COMMIT_AUTO_MINUTES	10	Intervalle du commit temporel automatique
SEUIL_COMMIT_BLOCS	50	Nombre de blocs modifiés déclenchant un commit immédiat
FENETRE_SEUIL_MINUTES	5	Fenêtre glissante pour le comptage du seuil

■ Tous ces seuils sont exposés dans un fichier de configuration admin. Un serveur puissant peut les augmenter, un serveur modeste les réduire.

9. API Publique de la Lib

9.1 Interface Principale

```
// Rollback composable WorldMemory.rollback() .at(centrePos) .radius(10)
.chunkMode(DELTA_EXPLOSIONS_ONLY) .entityMode(RESTAURER_MOBS)
.itemMode(RESTAURER_CONTENEURS) .nbtMode(COMPLET) .execute(world); // Snapshot
manuel WorldMemory.snapshot(world, centre, rayon); // Purge
WorldMemory.purge(chunkPos).administrative(adminUUID);
WorldMemory.purge(chunkPos).conditionnelle(PurgePolicy.parAge(30));
```

9.2 Système d'Événements

```
WorldMemory.beforeRollback(event -> { /* animation, particules... */ });
WorldMemory.afterRollback(event -> { /* log, audit... */ });
WorldMemory.onSnapshotExpired(event -> { /* game over, puzzle... */ });
WorldMemory.onPortalRestored(event -> { /* relier le portail... */ });
WorldMemory.onRollbackDegraded(event -> { /* crash détecté, effet glitch */ });
```

9.3 Enregistrement d'Explosifs Tiers

Les mods tiers peuvent enregistrer leurs propres entités explosives pour qu'elles soient trackées par la lib :

```
WorldMemory.registerExplosive( new Identifier("monmod", "super_bombe"),
ExplosiveConfig.builder() .puissance(8.0f) .fuseEstime(60) // ticks .build() );
```

■ Cette API sera finalisée une fois le cœur de la lib stabilisé. Les mods tiers non enregistrés ne seront pas trackés.

10. Problèmes Connus et Décisions

Tous les problèmes identifiés durant la conception et les décisions retenues :

Priorité	Problème	Solution retenue
RÉSOLU	Liquides (eau/lave)	Rollback comme un bloc classique via NBT. Pas de traitement spécial nécessaire.
RÉSOLU	Blocs à gravité (sable, gravier)	Non trackés intentionnellement. Bug accepté pour les performances. Si rollback, ils reviennent — duplication possible mais acceptable.
RÉSOLU	Mobs dans la zone	Capture NBT complète avant disparition UUID. Resupmon à la position originale avec équipement et état.
RÉSOLU	Portails inter-dimensions	Blocs restaurés avec NBT. Reliaison laissée au joueur ou au mod via événement onPortalRestored.
RÉSOLU	Modifications pendant le cast	Zone gelée pendant le rollback (configurable : FREEZE, NO_WARNING, CANCEL_IF_MODIFIED).
RÉSOLU	Structures générées vs constructions	ChunkGenerator pour le terrain + StructureManager pour les structures. Les deux appelés pour reconstituer l'état seed complet.
RÉSOLU	Spam du scan de propagation	Cooldown configurable entre deux scans (défaut 200ms). Cap sur les déclenchements par seconde.
FUTUR	Explosifs de mods tiers	API d'enregistrement prévue. Non implémentée avant stabilisation du cœur.
FUTUR	Crash pendant une chaîne	État CRASH_DETECTE au redémarrage. Post-snapshot pris après reboot. Rollback marqué non fiable (estFiable=false).

11. Roadmap de Développement

Phase 1 — Fondations (Couche Stockage)

- Objet store Git-like : hash SHA-1, déduplication, GZIP
- Génération de chunk depuis seed (ChunkGenerator + StructureManager)
- Comparaison delta vs seed
- Commit automatique temporel et par seuil
- Tests de performance et de volumétrie

Phase 2 — Historique et Tracking

- Stack temporel par chunk
- Système de causes et chaîne de responsabilité
- Flagging des entités explosives vanilla
- Algorithme de propagation en graphe (chaîne TNT)
- Capture NBT mobs avant disparition UUID
- Tracking spawnpoints joueurs

Phase 3 — Rollback

- Rollback par niveau (Chunk → Bloc → Entité → Item → NBT)
- Modes SEED_ORIGINAL, ETAT_PRECEDENT, DELTA_EXPLOSIONS_ONLY
- Règles de cohérence inter-niveaux
- Principe de Schrödinger pour les coffres
- Système de freeze de zone

Phase 4 — Purge et Stabilisation

- Purge administrative avec audit log
- Politiques conditionnelles composables
- Gestion des crashes (état CRASH_DETECTE)
- Throttling et groupage d'explosions
- Configuration admin exposée

Phase 5 — API Publique et Extensibilité

- API rollback fluente et composable
- Système d'événements (before/after rollback, etc.)
- API d'enregistrement d'explosifs tiers
- Documentation développeur
- Mod de magie de référence comme premier consommateur

12. Résumé Exécutif

WorldMemoryLib résout un problème universel dans l'écosystème de mods Minecraft : la mémoire et la restauration du monde. Aucune lib native n'offre ce niveau de granularité et d'efficacité côté mod (CoreProtect existe côté plugin Bukkit mais pas côté mod natif).

Le design s'inspire directement de Git pour le stockage (déduplication par hash, compression, delta vs seed) et propose une API composable à quatre niveaux permettant des usages allant du rollback chirurgical à l'effacement total d'une zone.

La philosophie centrale — aussi cruelle que miséricordieuse — garantit que la lib n'impose pas de décision éthique : elle expose des outils puissants et laisse le mod consommateur décider de leur usage.

Stockage	Git-like (CAS + GZIP + ZSTD + delta vs seed)
Granularité	Chunk → Bloc → Entité → Item → NBT
Rollback	3 modes × 5 niveaux × événements
Purge	Normale / Administrative / Conditionnelle
Performance	Groupage, throttling, priorités, background thread
Extensibilité	API explosifs tiers, événements, configuration
Fiabilité	Gestion crash, rollback dégradé explicite